Computer Science
25COC251
F225694

Drug-Target interactive Predictor

by

Daniel R. Smith

Supervisor: Mohamad Saada

Department of Computer Science
Loughborough University

January 2026

Contents

Chapter 1

Introduction

1.1 Project Brief

1.1.1 Motivation

The pharmaceutical industry faces high costs and long development cycles in drug discovery, partly due to the challenge of identifying whether a novel compound can bind to a specific protein target. Machine learning (ML) provides a promising pathway for automating and accelerating this process by learning patterns from large bioactivity datasets.

1.1.2 Objective

The aim is to develop a predictive machine learning system that will determine the likelihood of binding between a new drug-like molecule and a protein target in the early stages of the process. Hopefully, reducing the long development cycles the pharmaceutical industry faces in drug discovery by reducing the sample size early on.

1.1.3 Computational Approaches in DTI

Drug-Target-Interaction was primarily tested during wet lab experiments but the first *in silico*, computational approach, took place in 1982 with molecular docking of 3D structures (**?**). A later advancement to this was the breakthrough of using machine learning models, which have become increasingly more popular since the early 2000's, being able to autonomously recognize patterns and relationships from data. Ever since its introduction there have been many attempts to use machine learning but real progress was made in 2018 (**?**) when advanced deep learning models were created and tested, able to effectively learn representations from drug and protein sequence data.

1.1.4 Process

Data Acquisition

To extract and process relevant bioactivity data from BindingDB, a public database containing thousands of chemical compounds and protein target binding data.

Feature Engineering

Converting Chemical Compounds to molecular footprints using RDKit and encoding protein targets to numerical vectors using ProtBERT. These processes differ because the biological information is encoded in completely different forms, thus requiring domain-specific feature engineering methods.

Model Development, Training & Evaluation

The Neural Network will be developed and trained on the dataset to learn the underlying relationships that cause binders and non-binders. The model will be evaluated on validation metrics like AUC-ROC, AUC-PR, Precision@K and the Enrichment Factor. Based on these techniques, the Neural Network will be iteratively refined to reduce the error rate and improve accuracy.

Develop User Interface

The Website for this project will be developed using the react framework to speed up the development process and to allow for easy modifications. The user interface will be designed to be simple, easy to navigate and only contain necessary information relevant to the workflow. I want to display the result once the result is defined and the reasons why, history of comparisons made and the results, information of the drug and the protein and to give the user the ability to compare multiple chemical compounds against 1 protein to mimic wet lab work.

Tech Stack

Component	Technology / Library
Dataset	BindingDB
Programming Language	Python
Data Processing	pandas, NumPy, scikit-learn
Drug Molecule Feature Engineering	RDKit (molecular fingerprints, SMILES parsing, descriptor extraction)
Protein Feature Engineering	ProtBERT (protein embeddings using transformer-based model)
Neural Network Framework	PyTorch (model definition, training, and evaluation)
Model Evaluation	Scikit-learn
User Interface	React (frontend framework for model interaction and visualization)
Backend API	FastAPI (model serving and communication with frontend)
Version Control	Git / GitHub

Table 1.1: Technologies and tools used in the project.

1.2 Feature List

This is where I will show you all the features I wish to include within my project and what part they play in the project.

1.2.1 Required

Compare a single drug molecule to a single protein target

This will be the first and most basic functionality of the product as it will allow the user to search the ChEMBL database for a single drug molecule to test against a single protein target.

Compare multiple drug molecules to a single protein target

I will achieve this by allowing the user to multi-select drug molecules or selecting a drug based on its standard type and relation to filter it into groups. This will give the user a closer feel for what wet-lab work could achieve.

Displaying previous tests

Having a section which displays what the user had previously tested is important as it would reserve computational resources for new combinations to compare, preventing the user from re-using the neural network to repeat a binding.

1.2.2 Nice-to-Have

Produce indirect links into how they bind

Once the neural network has run and the result has been displayed to the user I would try and also include a brief explanation into why the network predicted what it predicted.

Displaying molecule and Protein information

Retrieving this relevant information from the ChEMBL database so the user can see more information about the compound-protein pair they have selected.

1.3 Scope & Limitations

1.3.1 Project Scope

To ensure a complete project and to counter issues that may spring up i will ensure that each milestone reached will provide a submittable piece of work that can be a good base to work off for the next stage:

Predicting drug-target interactions

Developing model capable of making predictions of drug-target interactions by classifying them into binding/non-binding.

Using neural networks for prediction

Implementing a training a neural network architecture to learn relationships between molecule and protein features, producing consistent and reliable predictions.

Including both chemical compound and protein sequence features

Extracting and integrating molecular descriptors or fingerprints for chemical compounds and sequence-based or embedding-based features for protein targets.

Implementing a web-based UI for result visualization and comparison

Creating a simple, React-based web application to visualize prediction results, allow compound-target comparisons, and provide basic interpretability.

1.3.2 Project Limitations

Here i will explain what is retracting from the project and preventing it from being as good as it could possibly be.

Data limitations

Due to the nature of the project i am resitricted to only using publicly available datasets (e.g. BindingDB, ChEMBL) that may have missing or biased data from their producers. Also the data set might not be complete missing vital information which could improve the models success rate.

Model limitations

Neural networks might not capture rare interactions or out-of-distribution compounds. Also neural networks don't take the structure of each molecule into account, i could also use a Graph neural network (GNN) however that gives me greater limitations computationally and data wise.

Computational constraints

I am only limited to what i have exposure to and the model will be trained on my personal laptop so my computational resources may not be representational of what big pharma companies have access to, producing a considerable gap into what the industry could be exposed to.

UI limitations

This is only a simple prototype so i'm not optimizing it for production scale but will make it suitable for a small team to use.

1.3.3 Assumptions

For this project i am assuming the only people using it will have sufficient prior knowledge within chemistry to use this product. I am assuming the data within ChEMBL is accurate and up to date.

1.4 Workflow

In this section i would like to display a visual representation on how i will take this project from start to finish.

1.4.1 System Overview

The system predicts potential interactions between chemical compounds and protein targets using a trained neural network. It processes input data, extracts numerical features, performs predictions, and displays the results through a web interface for comparison and interpretation

Component	**Description**
Data Collection & Preprocessing	Collecting compound and protein data from public databases such as BindingDB, and filtering it based on the required parameters.
Feature Engineering	Converting molecular structures and protein sequences into numerical feature representations such as fingerprints & embeddings
Model training and Evaluation	Training a neural network on the processed dataset to pickup on relationships and evaluating the performance with metrics such as AUC-ROC and Precision@K
User interface	Displays the result, history and allows the user to customize their comparisons. Built with react for responsiveness and ease of use

Table 1.2: System components and their descriptions.

1.4.2 Workflow Diagram

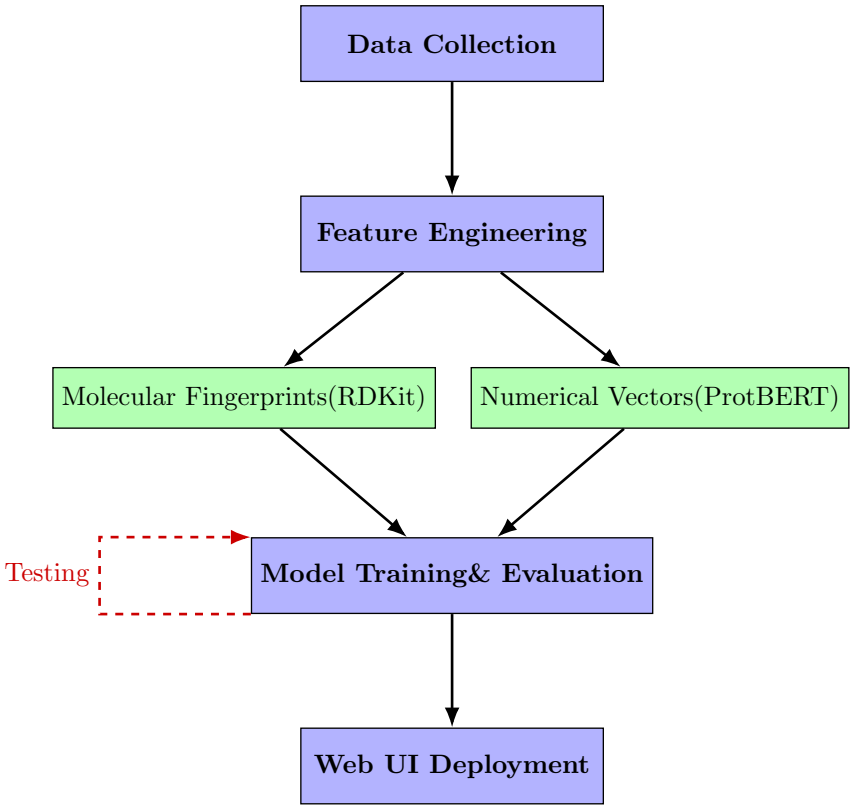

Figure 1.1: Top-down project workflow illustrating parallel feature engineering and iterative model refinement.

1.4.3 Project Timeline

Use of a Gantt chart to keep on top of things.

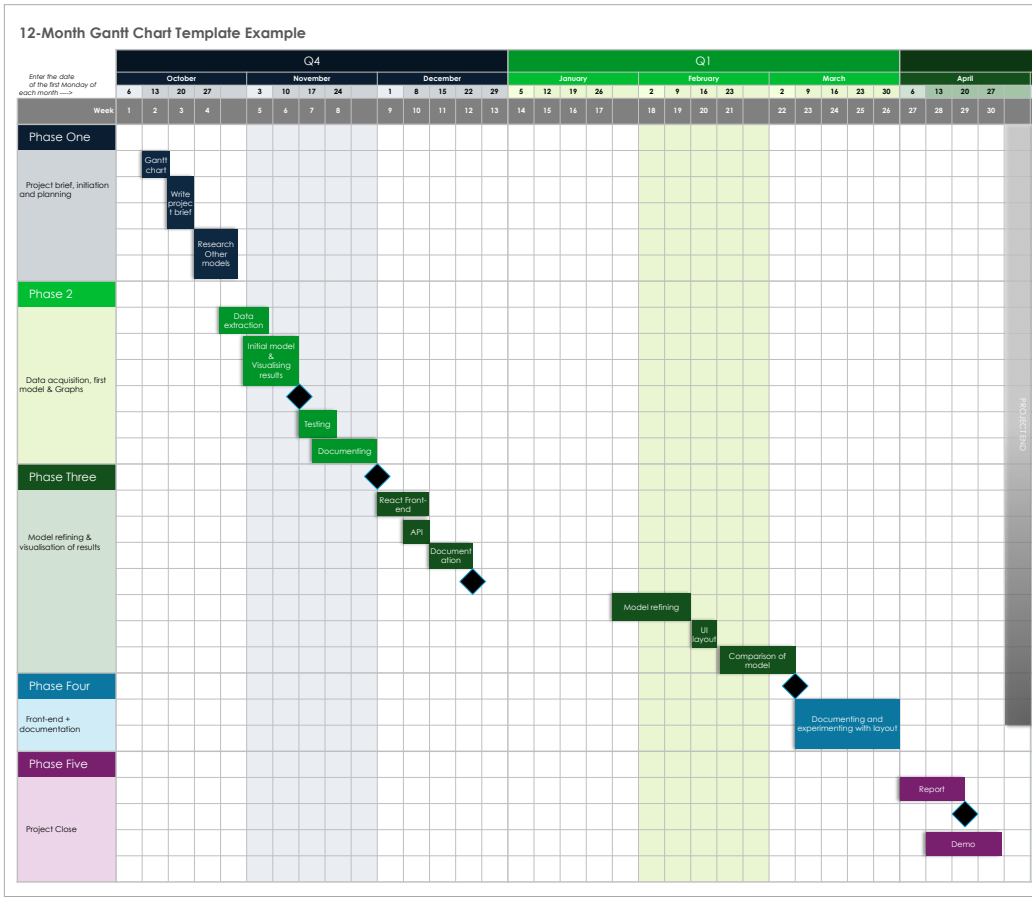

Figure 1.2: Project Gantt chart showing major milestones and development phases.

Chapter 2

Background Research

This chapter reviews the current state of research and available tools in the field of drug-target interaction prediction. The review is divided into (i) foundational concepts, (ii) current machine-learning approaches, (iii) existing prediction tools, and a (iv) gap analysis highlighting the need for the proposed project.

2.1 Initial concept

Drug discovery is the process through which potential new therapeutic entities are identified, using a combination of computational, experimental, translational, and clinical models (**?**). Drug-Target Interaction (DTI) prediction is a critical aspect of drug discovery, as it helps in identifying potential drug candidates that can interact with target proteins to exert their desired therapeutic effects (**?**). A drug is a chemical compounds which cause physiological changes in the body when consumed, injected or absorbed. A target, also known as a biological target, is a structure located in an organism that is recognized or bound by other substances such as ligands or drugs and can be acted upon by a drug or other targeted molecules (**?**). The pharmaceutical industry faces mounting financial pressures and long development cycles in drug discovery which creates a huge challenge when discovering new drugs, especially when they are need in short time spans, as seen and experienced with COVID-19. One way to combat these challenges is to utilize machine learning to potentially automate the initial screening process, reducing both equipment and labour costs. This approach is very valuable in the initial stages of drug discovery when the sample size is extremely large and can be accurately whittled down by an algorithm trained to predict interactions based on known patterns.

2.2 Foundations of DTI Machine Learning

In the beginning of DTI using machine learning, methods included similarity-based approaches. This is where drugs with similar structure are assumed to interact with similar proteins (Find citation for this). Feature-based machine-learning models are much more refined and rely on more scientific knowledge compared to just their structure, such as fingerprints or protein sequence features. These come in 2 different ways, Regression - where the model predicts continuous binding affinity and Binary classification - where affinity values put into thresholds, active or inactive.

2.3 Deep learning models in DTI

Deep Learning has become a strong way to determine interactions, due to its ability to learn high dimensional representations from raw data. There are many approaches for deep learning architectures, for example the use of fully connected neural networks (FCNN), (**?**)(**?**) which is a layered network that connects every neuron in one layer to every neuron in the next layer and evaluated using Accuracy, Precision, Recall, AUC-ROC, and AUC-PR.

2.3.1 Sequence-Based Models

A Sequence based model encodes raw data, Drug SMILES and Protein amino acid sequences, directly. Recurrent Neural Networks (RNN) and Convolutional Neural Network (CNN) are used to learn patterns in local chemical environments or amino-acid motifs. Models like DeepDTA (**?**) displays impressive predictive performance using sequence-only architectures.

2.3.2 Graph-Based models

Graph Neural Networks (GNNs) take as input the structure of molecules as graphs of atoms, learning structural information which, to sequence based models, are difficult to learn. (**?**) utilized this as part of their algorithm using Message Passing Neural Network (MPNN) to encode drug sequences as a part of a greater system. An MPNN can extract more detailed relational features, for improved modeling of chemical structure.

2.3.3 Hybrid Networks

A Hybrid network is the combination of multiple neural network architectures to capture complimentary inormation to improve the learning rate of the model. We will also use Yang's (**?**) whole hybrid model, as well as an MPNN to encode drug sequences and a

Convolutional Neural Network (CNN) to encode protein sequences. To generate the other prediction it uses a High Order Graph attention Convolutional Network (HOAGCN), Which makes the prediction based on the structure of the drug molecule and target protein. All these combined make for a much more meaningful prediction producing accurate results.

2.4 Binary Classification in DTI

There's lots of research and studies that have been done using binary classification for the interaction task, this is where the model outputs whether it will (1) or it won't (0). This has become increasingly more popular as it simplifies the learning objective allowing the model to spend more computational power on classification metrics.

2.4.1 Research Papers Using Binary Classification

A strong model which uses this type of classification is DeepDTA (**?**), which applies a threshold to continuous affinity values (IC50 or Ki) and classifies the interactions as active or inactive. Although the study also explored regression, it showed that the binary classification improves model stability and reduces noise introduced by inconsistent conditions. Another strong case is produced in Talukder (**?**), who uses explicit affinity binarization combined with MACCS fingerprints and protein sequence embeddings to train a neural network that outputs a binary interaction score. This is supported by Zhan (**?**), who follows the same approach, emphasizing the robustness in binary prediction when affinity values are inconsistent.

2.4.2 Affinity Thresholds

The affinity thresholds used are very important because it determines the data the model ingests and whats its classified at. Having an off centered threshold can weight the dataset to one way even if it went in 50:50. In DeepDTA (**?**) they used the thresholds for evluation againts other datasets (Davis, KIBA). They used a pKd ¿= 7 as binding on the Davis dataset and on the KIBA dataset they used a KIBA score ¿= 12.1 as the threshold. This is pretty muhc used a golden standrda among the indsurty as later works reference DeepDTA for their threshold choices.

2.4.3 Negative Sampling

Some of the recent work that uses binary classification also highlight the importance of negative sampling during training. Since many DTI datasets tend to be weighted towards positive pairs, the data becomes skewed. Several recent models including DeepPurpose (**?**)

and other BindingDB approaches, address skewed data using negative sampling to generate synthetic pairs to avoid having an unbalanced training dataset.

2.4.4 Evaluation metrics

Commonly seen among these studies in binary classification, (**???**), they rely (i) Accuracy, how many observations were correctly classified (ii) Precision, How many positive predictions were actually positive (iii) Recall, how many positive cases did the model correctly predict (iv) AUC for either (i)PR, A chart that visualizes the Precision and Recall in a single graph or (ii)ROC, is a chart that visualizes the trade-off between the true positive rate (TPR) and the false positive rate (FPR).

2.4.5 Binary Classification Summary

Binary classification has become the basic method for contemporary DTI research because it can complete the tasks at machine-learning benchmarks but in a more simple way. This is why i have decided to pursue a model that works with binary classification and use classification based evaluation metrics.

2.5 Dataset & Feature Engineering

Recent research papers consistently take as input drug SMILES and proteins as Amino Acid Sequences from BindingDB's dataset (**?**), reinforacing the fact that this is the most appropriate source data in DTI. A few studies differ mainly in how they encode these inputs. For example the paper by Talukder(**?**) represented the drugs as MACCS fingerprints generated by RDKit, these are fast, interpretable and simple but are not very expressive making them easy to compute which can benefit me with limited computational power. Protein features are similarly derived from amino acid sequences through embedding frameworks such as ProtBERT. While datasets like Chembl are available, the frequent occurrence of BindingDB in recent literature could suggest that it's the current benchmark for DTI projects.

2.6 Common limitations

What seemed to be a common limitation was the lack of data on successful binds creating an imbalanced dataset, however some of these papers tried to combat this issue by taking data from two datasets (**?**) or manipulating the data through affinity harmonization & binarization (**?**) and using Polyloss framework (**?**), which is an addition to standard loss functions that allows us to precisely tune how the model handles our severe class imbalance.

2.7 Current products

2.7.1 SwissTargetPrediction

(?) (?) Is a website which allows the user to predict using a ligand-based approach built to combine different similarity measures, providing more accurate predictions. The method, as described in the 2013 paper, was trained on a set of 224,412 molecules known to interact with 1700 human proteins from the ChEMBL database.

For many bioactive molecules their primary target is unknown and even when it is, most have more than one target (Off-targets) that are not well characterized. Making predictions for these off-targets is important for (i)understanding side effects and (ii)drug repositioning - finding new uses for old drugs.

The method's main strength is its hybrid approach to similarity, combining multiple variables to discover unknown targets of similar structure. It uses and combines 2D Chemical Similarity: This uses FP2 chemical fingerprints to find structurally similar molecules, which is effective for identifying analogs(structurally similar molecules) within the same chemical series. (ii) 3D Shape Similarity: This uses Electroshape, which is a 3D method that compares molecules based on their shape, atomic partial charges, and lipophilicity. This is powerful for finding active molecules with different chemical structures.

These two scores are weighted and combined using a multiple logistic regression model. The authors demonstrated that while 2D fingerprints alone perform well on biased datasets, the combined 2D/3D approach is significantly more accurate in the more realistic scenario of predicting targets for novel molecules that do not belong to a known chemical series.

The interface is plain and a bit scattered but provides an easy use, you select a species (i)Homo sapiens, (ii)Mus musculus, (iii)Rattus norvegicus, and then choose a drug molecule through given example, SMILES representation, or drawing the molecule. which is creative and more advanced way to really test new drug compounds and structures.

2.7.2 DINIES

(?) (?) (Drug-target Interaction Network Inference Engine based on Supervised Analysis) is a web tool that predicts potential interactions based on drug data and omics-scale protein data, allowing the user to use whatever data as input as long as it's represented accordingly. First published in 2014 it uses machine learning and predicts using the "guilt-by-association" principle, similar drugs are likely to interact with similar proteins.

The methodology is based on using Kernels to convert (i)chemical structure (ii) side-effect and (iii) Protein sequence profiles to separate matrices. The server combines the matrices and in its simplest mode will just average them together to create a single combined drug

similarity matrix.

However this foundational machine learning method has become very limited compared to other products as it over relies on features and it only learns simple mapping, not complex mapping or non-linear relationships from raw data. Furthermore, the UI is not designed for simple queries as it requires the user to format and upload their own data matrices, creating a high barrier to entry for most researchers.

2.8 Gap analysis

After a detailed review of Drug-Target interaction products and papers, from foundational similarity-based methods to modern deep learning models, three gaps in the current landscape have appeared: (i) **SOTA vs Usability Gap**, gap between modern, cutting-edge research models and tools accessible to the public (ii) **Novelty problem**, similarity-based models will struggle when given novel drugs or targets which have no known analogs (iii) **The Data Imbalance gap**, older datasets were imbalanced and some using conventional loss functions to counteract but they can turn out biased, resulting in poor performance.

My proposed predictive ML pipeline will be designed to directly address these three gaps, (i) It will deliver SOTA accuracy by using modern neural network architectures to learn representations reducing the novelty issue (ii) utilizing the PolyLoss function to force the model to focus on hard-to-find positive interactions to address the data imbalance challenge (iii) diminishing the usability gap by connecting powerful backend to a simple UI, displaying results and information in an accessible way, a feature missing from most complex models.

Chapter 3

Data Pipeline

3.1 Introduction

This chapter provides a detailed overview of how the data was sourced, processed and prepped for two inputs, proteins and drugs, turning them from raw data to computationally readable for the model.

3.2 Data Collection and Curation

Data is an extremely important part of this project, and it's important to gather a sufficient amount of scientifically accurate data so the model can be the most accurate as possible. This section will define how the data was sourced and manipulated to reach its final representation.

3.2.1 Primary Data Source

I extracted the initial bulk of my data from BindingDB (**?**), which is a public database holding experimentally measured binding affinities between small molecules and proteins. BindingDB serves as a critical resource to a diverse range of applications, including development of artificial intelligence models. The database has grown significantly, holding information on 2.9 million binding measurements for 1.3 million compounds. Data can be extracted through its web-services or downloaded directly. Due to the nature of the project, the initial phase begun with a local download of two raw data files to facilitate data cleaning and binarization:

- `BindingDB_all.tsv` file containing millions of interaction records

- `Sequences.fasta` file containing the amino acid sequences for all protein targets in the database.

3.2.2 Data Merging and Cleaning

The raw data from BindingDB didn't contain protein sequences making it unsuitable for feature engineering, so the first step after downloading all raw data was to unify the interaction and protein sequence data. Using the proteins name as a common key, the `.tsv` and `.fasta` file were merged creating a unified dataset where each row contained **Drug SMILES**, **Target Sequence**, **Binding Affinity Value** and the **Interaction classification**.

To generate the binding affinity value, we unified multiple affinity sources (Ki, Kd, IC50 and EC50) into a singular NanoMolar value and converted them into a log-scaled representation *pAffinity* similarly to He (**?**). This compressed the range of BindingDB affinity values into a more stable scale, improving the models behavior when learning.

During the merge, the data was also cleaned to remove duplicate interactions or any rows with NULL values and was filtered for Homo-Sapien targets only. The result was a completely unified table containing the necessary information to continue with feature engineering.

3.2.3 Ligand Identifier Normalization

Ligand identifiers provided by BindingDB corresponded to experimental compounds instead of approved and recognized drug names. This made it difficult to use the front end as the identifiers were very lengthy and chemically descriptive. To address this, the identifiers were normalized to be in simpler forms while being retained to ensure traceability back to the original BindingDB database.

Listing 3.1: Ligand name normalisation from BindingDB identifiers

```
def normalize_ligand_name(self, name: str):
    if name is None or pd.isna(name):
        return None
    name = str(name).strip()
    if not name:
        return None
    parts = [p.strip() for p in name.split("::") if p.strip()]
    if not parts:
        return None
```

```
    parts = sorted(parts, key=len)
    for p in parts:
        if not re.search(r"[=#\[\]\(\)]", p):
            return p
    return parts[0]
```

A snippet of the raw `.csv` file is shown in Table **??**.

Table 3.1: Snippet from `bindingdb_all.csv` (Raw)

...	Ligand SMILES	**Target Name**	Ki (nM)	...
...	`CC(=O)Oc1...`	**Thymidine kinase**	50.0	...
...	`C[N]1C=C...`	**Thymidine kinase**	12000.0	...
...	`CC(C)(C)OC...`	**ABL1 kinase**	75.0	...

The `.fasta` file was parsed into a key-value dictionary to map protein names to their sequences, as shown in Table **??**.

Table 3.2: Example of Parsed `binddbtargetsequence.fasta` Data

Key (from FASTA Header)	Value (Sequence)
Thymidine kinase	`MPK...` (sequence truncated)
ABL1 kinase	`GKL...` (sequence truncated)
...	...

These two sources were merged on the target name. After merging, cleaning, and binarization the final unified dataset was created and a snippet shown in Table **??**.

Table 3.3: Snippet of Final Merged and Processed Dataset

`drug_smiles`	`target_sequence`	`target_name`	`affinity_nm`	`interaction`
`CC(=O)Oc1...`	`MPK...`	**Thymidine kinase**	50.0	**1**
`C[N]1C=C...`	`MPK...`	**Thymidine kinase**	12000.0	**0**
`CC(C)(C)OC...`	`GKL...`	**ABL1 kinase**	75.0	**1**

3.2.4 Affinity Type Justification (Ki, Kd, IC50)

BindingDB already provides some experimentally measured binding affinity types, such as *Ki*, *Kd* and *IC50*. Although these quantities are related, they do not supply us with meaning

and are not suitable to begin our classification of the dataset.

Definitions and Mechanistic Differences

Ki (inhibition) measures the binding between an inhibitor ligand to an enzyme, which strictly reduces the catalytic activity of the enzyme. **Kd** (dissociation) also measures the binding equilibrium between a ligand and protein but in a more general, all-encompassing term to include all the dissociated components. While mechanistically similar to Ki, Kd describes the general tendency of a complex to dissociate into its constituent components. Both Ki and Kd are equilibrium constants governed by the same thermodynamic principles (**?**). **IC50** is short for inhibitory concentration 50%, meaning the concentration of inhibitor required to reduce the biological activity of interest to half of the uninhibited value. The simplicity and lack of assumptions required to generate these data make them useful but as it doesn't directly measure the binding equilibrium its a less accurate measurement than Ki or Kd. The relationship between IC50 and Ki is described by the Cheng-Prusoff equation, which shows that IC50 varies with substrate concentration even when Ki remains constant (**?**). **EC50** stands for effective concentration 50%, which refers to the concentration of drug at which 50% of its maximum effect is achieved. Making it more effective for experiments wanting the dosage to be quantified even when the enzymes activity doesn't reach the 50% threshold causing IC50 to be undefined. It reflects functional activity rather than direct binding affinity. EC50 measurements incorporate complex factors including receptor reserve, signal transduction efficiency, and cellular response mechanisms (**?**).

Justification for Ki Selection

To evaluate which affinity type is most appropriate for my model, we analyzed the BindingDB dataset across four key criteria: mechanistic validity, data quality, measurement reliability, and data availability. **Mechanistic Validity:** Ki and Kd represent true equilibrium binding constants that are substrate-independent and thermodynamically well-defined. In contrast, IC50 and EC50 are functional assays whose values depend on experimental conditions and do not directly measure binding affinity. For a machine learning model intended to learn binding relationships, using equilibrium constants provides a more robust and interpretable basis (Figure **??**A). **Data Quality:** After reviewing the graphs generated from the raw bindingDB file, Ki has the lowest outlier rate which indicates more consistent experimental measurements compared to IC50 for Kd. Higher consistency is better for training the model as its reducing noisy and inconsistent labels from the dataset (Figure **??**B). **Data Availability:** While IC50 measurements are more abundant in BindingDB (47,043 entries), Ki still provides 16,796 high-quality measurements. This is plenty to develop and train ma-

chine learning models as modern deep learning models have shown its more valuable to have high quality data than a higher quantity (**?**). **Cross-Validation with Correlation Analysis:** To confirm the relationships between affinity types, a correlation heatmap was generated across all measurements in the raw BindingDB dataset (Figure **??**). The high correlation between Ki and Kd (r = 0.94) confirms their mechanistic similarity as equilibrium constants. The moderate correlation between Ki and IC50 (r = 0.81) reflects IC50's dependence on experimental conditions, while the weaker correlation with EC50 (r = 0.55) highlights its distinct mechanistic basis as a measure of functional activity rather than binding affinity.

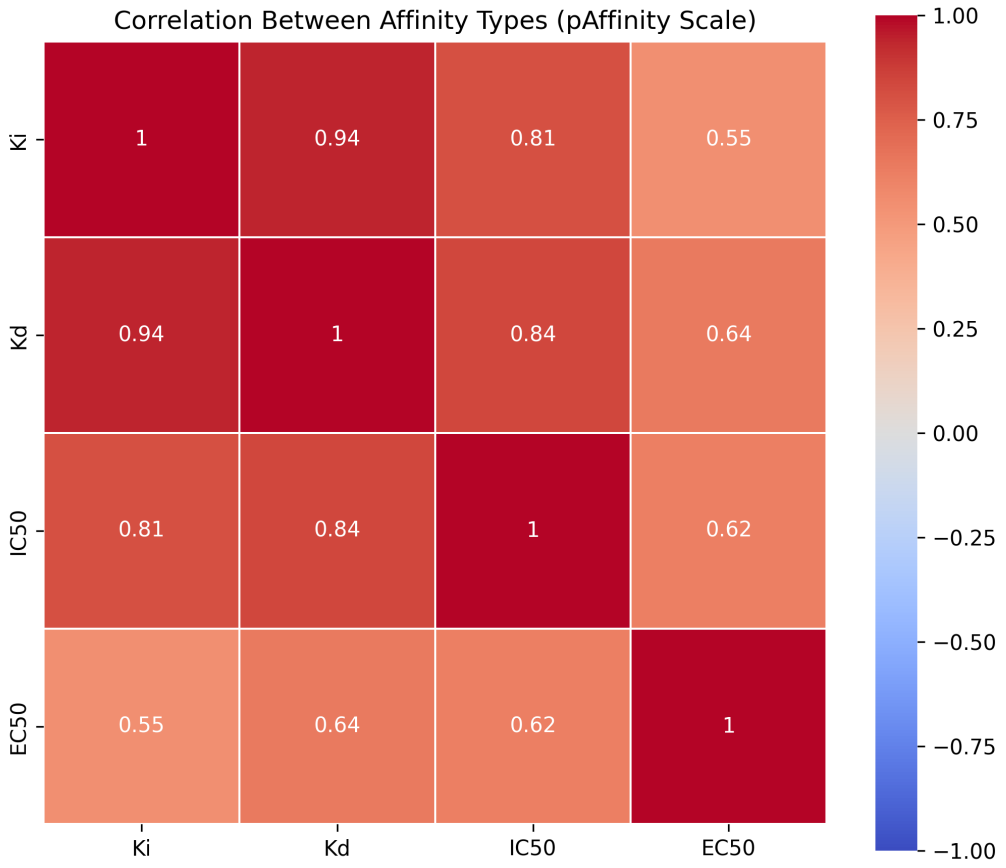

Figure 3.1: Heatmap showing the correlation between the different affinity values to show similarities in the values given

Based on this analysis, **Ki was selected as the primary affinity measurement** followed by Kd then IC50 as Kd will provide more accurate data than IC50 but its not

present enough to be used initially. Ki provides the perfect balance of mechanistic validity (true equilibrium constant), data quality (lowest measurement variability), and sufficient quantity (16,796 measurements) for reliable DTI classification.

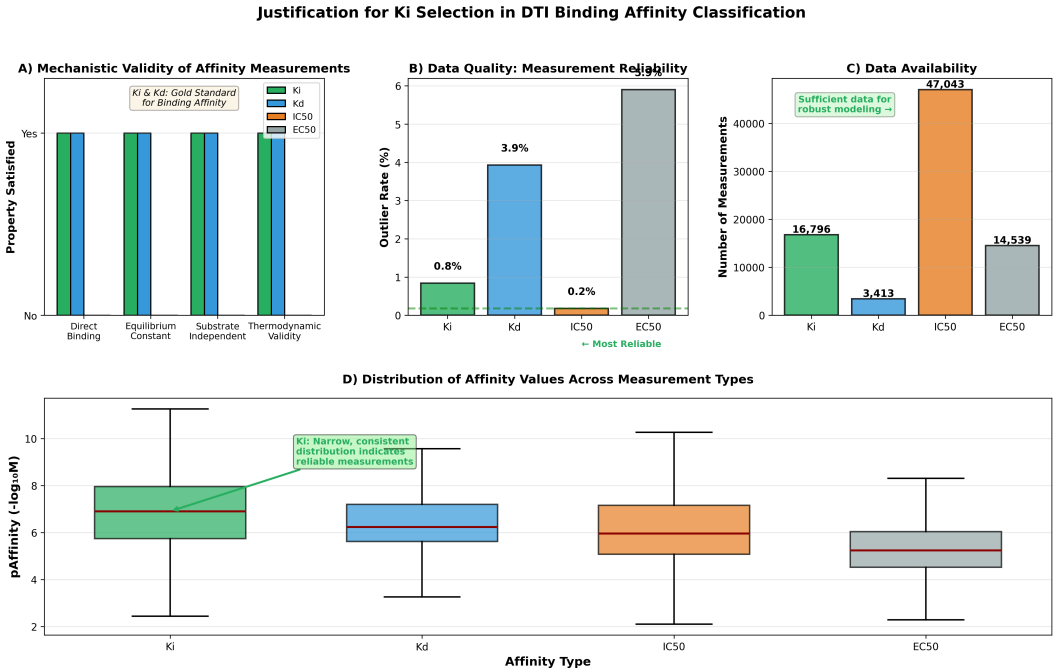

Figure 3.2: Comprehensive justification for Ki selection. (A) Mechanistic validity comparison showing Ki and Kd satisfy all criteria for true binding measurements. (B) Data quality assessment via outlier rates. (C) Data availability across affinity types. (D) Distribution of pAffinity values showing Ki's consistent measurement characteristics.

3.2.5 Data Binarization & Negative Sampling

It's very important to classify the data accurately with meaningful representations and thresholds. From the BindingDB dataset we utilized the columns 'Ki' & 'Kd' and set thresholds on this value to determine the score binder or non-binder. Instead of using raw nanomolar(nm) values, the binding affinities were transformed into their log-scaled form using equation **??**:

$$\text{pAffinity} = -\log_{10}(\text{Affinity (M)}) \tag{3.1}$$

where the raw nanomolar (nM) value is first converted to molar (M) units using equation **??**:

$$\text{Affinity (M)} = \text{Affinity (nM)} \times 10^{-9} \tag{3.2}$$

This representation is widely used in the industry and literature (**????**) due to its biological interpretability and numerical stability.

A common challenge with data in DTI is the insufficient number of true negative data as databases are biased towards successful binding experiments. A basic solution to this is random negative sampling, which pairs drugs and proteins at random and assuming they do not interact. However, this can introduce a high rate of false negatives and degrade model performance, which is highlighted as an issue of *low confidence negative samples* by (**?**). Methods like fuzzy-rough approximation (**?**), select negatives by degree rather than at random providing a better alternative for this type of sampling.

To counter this issue, we defined two binding thresholds and used negative sampling with care by following the 'safe-margin' practice described by Bai, (**?**). Therefore we classify pairs with a pAffinity value > 7 as a binding interaction (label = 1), and pairs with pAffinity value < 5.3 as non-binding (label = 0).

3.2.6 Implementation of Affinity Transformation and Label Assignment

Listing 3.2: Affinity transformation and binary label assignment

```
df_chunk = df_chunk[df_chunk['affinity_value_nm'] > 0].copy()
# Convert to pAffinity (-log10)
df_chunk['p_affinity'] = -np.log10(df_chunk['affinity_value_nm'] * 1e-9)
# Create Binary Labels (-1 = invalid, 0 = non-binder, 1 = binder)
df_chunk['interaction'] = -1
df_chunk.loc[df_chunk['p_affinity'] > BINDER_THRESHOLD_P, 'interaction'] = 1
df_chunk.loc[df_chunk['p_affinity'] < NONBINDER_THRESHOLD_P, 'interaction'] = 0
# Keep only valid 0 or 1
df_chunk = df_chunk[df_chunk['interaction'].isin([0, 1])]
```

This approach generates an area between the thresholds, which represents an ambiguous *grey area*, a zone where the interactions are too week to be a binder but too strong to be a clear non-binder. To increase performance of the model we only want to train it on the clearest signals, so all pairs inside the *grey area* are removed from the loaded dataset shown in Table **??** and a distribution graph visualising the grey area is shown below in Figure **??**.

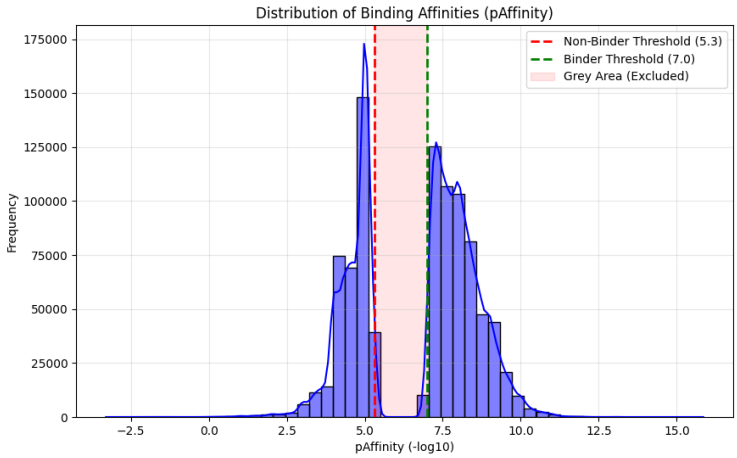

Figure 3.3: Distribution of pAffinity values in the filtered dataset. The red shaded region represents the *Grey Area* ($5.3 \leq$ pAffinity ≤ 7.0), where data points were excluded to ensure high-confidence labels.

After the binarization and cleaning it generated the figures seen below in Table **??**.

Table 3.4: Final Class Distribution after Thresholding

Class Label	**Interaction Type**	**Threshold Criteria**	**Count**	**Percentage**
1	Binder (Positive)	pAffinity > 7.0	557,959	60.25%
0	Non-Binder (Negative)	pAffinity < 5.3	368,066	39.75%
Total			**926,025**	**100%**

3.3 Feature Engineering

This section outlines the Feature engineering process to turn raw chemical and biological data into machine readable formats. Drug SMILES and Target Sequences must be encoded into fixed-size numerical vectors as neural networks cannot process the raw text representations.

3.3.1 Drug Representation

Drug SMILES being represented as variable length strings are unsuitable for the model to ingest. To address this issue drug SMILES were encoded into MACCS keys, a 167-bit binary fingerprint, where each bit represents the presence of a different chemical substructure.

3.3.2 RDKit

The open-source Cheminformatics toolkit RDKit performed *Descriptor Generation* by parsing SMILES strings into *Mol objects*, translating complex chemical structures into a fixed length vector (MACCS Keys) that the model can use to learn patterns for each drug & protein pair. (**?**) As a vector notation:

$$\text{MACCS} = [0, 0, 1, 0, 0, 0, 0, 0,\ 1, 0, 0, 0, 0, 0, 0, 0,\ 1, 0, 1, \dots] \in \{0, 1\}^{167}.$$

3.3.3 Protein Representation

The Proteins are represented as amino acid sequences of varying lengths, often exceeding thousands of characters. To normalize the sequences, so they're suitable for the neural network, the variable-length sequences were converted into fixed-size continuous embeddings.

3.3.4 ProtBERT

To ensure the embeddings were as accurate as possible, ProtBERT was used as it's a pre-trained transformer model based off of the BERT architecture (**?**). ProtBERT understands the contextual and biophysical properties of the amino-acid sequences as its been trained on a huge database of protein sequences UniRef100 (**?**) containing 217 million sequences (**?**). By extracting the hidden states from the model through mean pooling, each protein sequence is condensed into a semantic vector embedding that fits the fixed-size ,*1024 floating-point values*, input and preserves the biological context, such as 3D structure or chemical polarity. (**?**)

Protein embedding (example, size 1024):

$$\mathbf{p}_{\text{ex}} \in \mathbb{R}^{1024} = [\,0.320856,\ -0.685886,\ -0.652373, \dots,\ -0.344068]$$

3.4 Data Preparation

Following feature engineering, the processed data was split into 3: *Training, Validation* and *Testing*. This is an important step to prevent overfitting of the model on the dataset, keeping this randomized is key to the validity of the model.

3.4.1 Train-Validation-Test Split Methodology

To ensure reproducibility the dataset was split similarly to (**??**) at a ratio of 65:20:15. To make it a stratified random sampling approach, a fixed random seed (random_state = 42) was used to make the sampling procedure deterministic, meaning each run produces the same

split. This is important for reliable model evaluation, comparison between experiments, and debugging. The splitting was split into two stages:

- **Primary Split** - The merged dataset was split into a 65% for Training and 35% for a Secondary set.
- **Secondary Split** - the secondary set of data was split to 57.14% and 42.86% to obtain the overall split of 20% for validation and 15% for testing.

Table 3.5: Dataset Partitioning for Model Training and Evaluation (65:20:15 Split)

Subset	**Split %**	**Sample Count**
Training	65%	601,916
Validation	20%	185,205
Testing	15%	138,903

3.4.2 Justification of Split Ratios

While an 80:20 ratio between training and testing is common (**??**), A higher validation set of data at 20% was used for more accurate estimates of model performance during training. This can allow for better hyperparameter tuning so the model can be tweaked to optimize performance on the unseen test data. This specific split has been used among other deep learning studies that require rigorous hyperparameter tuning, such as in medical image analysis (**?**) and environmental prediction modeling (**?**). A large allowance to the validation set reduces the risk of overfitting before the final evaluation on the unseen test data.

Chapter 4

Model Design and Implementation

4.1 Model Overview

4.1.1 Task Definition

The objective of this work is to address the DTI prediction problem as a binary classification task.

4.1.2 Model Inputs and Outputs

The inputs and outputs were explained in detail in my data chapter **??** but I will briefly go over the inputs and outputs. The inputs going into the model are Protein embeddings and Drug embeddings with strong semantic meaning. The output is either a yes or no classification determined by thresholds mentioned **??**.

4.1.3 Learning Paradigm

The learning paradigm, how the model learns from the data, is supervised binary classification using PyTorch-based gradient optimization using Adam (**?**). The model learns to distinguish between binding and non-binding drug-target pairs based on labeled training data.

4.2 Problem Formulation

The prediction task is formulated as a supervised binary classification problem, where the objective is to conclude whether a given drug and protein pair produce a binding interaction. The data instances of drug molecules and target proteins are represented as fixed length numerical vectors where $\mathbf{d} \in \mathbb{R}^m$ denotes the representation of a drug molecule and $\mathbf{p} \in \mathbb{R}^n$ denotes the representation of a target protein.

The goal of the model is to learn a function $f : \mathbb{R}^m \times \mathbb{R}^n \to [0, 1]$, which maps a drug–protein pair $(\mathbf{d}, \mathbf{p})$ to a continuous interaction score $\hat{y}$ representing the predicted probability of binding: $\hat{y} = f(\mathbf{d}, \mathbf{p})$. To obtain a binary interaction label, the predicted score $\hat{y}$ is compared to the predefined decision threshold τ, which is a tunable parameter; the value of $\tau = 0.3$ is justified through data sensitivity analysis in Chapter **??**:

$$\hat{y}_{\text{class}} = \begin{cases} 1, & \text{if } \hat{y} \geq \tau, \\ 0, & \text{otherwise.} \end{cases}$$

The ground-truth labels $y \in \{0, 1\}$ are derived from experimentally measured binding affinity values and binarised using affinity thresholds described in Chapter **??**. Pairs exceeding the binding threshold are labeled as interacting, while pairs below the non-binding threshold are labeled as non-interacting.

Classification was selected over regression to improve robustness against experimental noise and to focus learning on more definitive binding interactions. This formulation aligns with the intended application of the system, where the primary objective is to determine whether a candidate drug is likely to bind a given target rather than to predict an exact affinity value.

The learning objective is, therefore, to minimise the discrepancy between the predicted interaction labels $\hat{y}$ and the true labels y in the training dataset, allowing the model to generalise to the unseen drug-target pairs.

4.3 Integration of Input Representations

Drug to Target interaction predictions requires the combination of two different data styles, drugs being small chemical structures and proteins being amino acid sequences. These representations differ in scale, semantic meaning, and dimensionality, making direct integration inappropriate.

To address this, the model uses a dual-input design, where drugs and proteins are processed independently through dedicated encoding pathways before being combined at a later stage once they become compatible. Allowing independent processing preserved the features

and semantic meaning for each drug and protein, and joint learning is enabled once combined together.

4.3.1 Drug Representation

Drug molecules are represented using MACCS fingerprints, which encode the presence or absence of predefined chemical substructures as a fixed length binary vector of 167 dimensions.

The fixed dimensionality makes it well suited for neural networks as it allows the model to learn patterns between specific substructures and binding behavior. This also makes it computationally efficient and reproducible across multiple experiments.

4.3.2 Protein Representation

Target proteins are represented using contextual embeddings created by the pretrained ProtBERT transformer model. Unlike traditional sequence-based embeddings, transformer-based embeddings capture bidirectional and context-dependent semantic meanings within amino acid sequences, reflecting underlying biochemical and structural properties.

Each protein sequence is mapped to a fixed-length vector through mean pooling to produce a representation suitable for neural network processing. Mean pooling provides a simple method for aggregating variable-length protein sequences while preserving global contextual information learned by the transformer. This approach enables the model to leverage rich semantic information learned from large protein databases, improving generalization to unseen targets.

4.4 Model Architecture

Due to the heterogeneous nature of Duel Inputs, the architecture is slightly modified from traditional neural networks. Separate encoding pathways are used to process each input to a usable, unified representation for interaction prediction.

Both encoders operate as fixed feature extractors and are not updated during model training. Learning is therefore concentrated in the prediction stage, reducing model complexity and improving training stability.

This modular design enables future extensions, such as introducing trainable encoder layers or alternative fusion strategies, without requiring changes to the underlying data pipeline.

4.4.1 Drug Encoder

The drug encoder is responsible for transforming molecular SMILES string into a numerical representation using MACCS (Molecular ACCess System) keys (**?**). The SMILES representation is parsed into an RDKit object, where MACCS fingerprint generation is applied producing a 167-bit binary vector.

This representation captures essential structural properties of small molecules while maintaining a compact and consistent dimensionality. In cases where a SMILES string cannot be successfully parsed, a zero vector is returned to preserve input dimensional consistency and prevent pipeline failure.

4.4.2 Protein Encoder

Protein sequences are encoded using contextual embeddings derived from a pretrained ProtBERT transformer model (**?**). Each amino acid sequence is tokenised and passed through the frozen ProtBERT model to obtain hidden states.

To obtain a single fixed-length representation per protein, mean pooling is applied across the sequence dimension of the final hidden layer. This produces a 1024-dimensional continuous embedding that summarises the biochemical and contextual properties of the entire protein sequence.

This is computationally expensive, so each result is cached locally using hash-based lookup for reduced overhead when a sequence has already been computed. This is very effective during dataset re-processing as some sequences are loaded directly from the disk.

As with the drug encoder, the protein encoder operates as a fixed feature extractor and does not participate in gradient-based learning during model training.

4.4.3 Prediction Network

The prediction network contains two hidden layers using ReLU activations for learning non-linear interactions between the drug and protein representations. The network also makes use of dropout and L2 regularisation to prevent overfitting, which is a common issue in models with limited training data and high-dimensional inputs. The network uses the sigmoid function only on the output layer to produce a continuous interaction score between 0 and 1. Here is a tale describing the architecture of the prediction network:

Table 4.1: Model Architecture and Training Hyperparameters

Parameter	**Optimal Model**
Architecture	
Hidden size	32
Activation	ReLU + Sigmoid (Output layer)
Batch normalisation	No
Dropout	0.2
Training	
Loss function	BCE
Optimiser	Adam
Learning rate	1x10*-3
Weight decay	1x10*-4
Batch size	512
Max epochs	1000
Regularisation	
Early stopping	No
LR scheduler	—

The design choices reflected here were informed through iterative experimentation, the full progression is documented in Chapter **??**.

4.4.4 Fusion and Prediction Layer

Once both inputs have been encoded, both representations are combined to form a joint input for the prediction model. The Binary MACCS fingerprint with dimensionality 167 and the ProtBERT embedding with dimensionality 1024, are concatenated into a single vector with dimensionality 1191 representing the drug-target pair.

This combined vector is then passed through the prediction network described above, and produces a binary classification based on the optimal threshold determined through experimentation.

4.5 Training Procedure

This section describes how the model parameters are optimised during training, a process that is important to how the model learns meaningful patterns and builds internal representations on the data it's exposed to.

4.5.1 Loss Function

The model is trained using the Binary Cross Entropy (BCE) loss function, defined as:

$$E = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)\right],$$

where N is the number of training samples, y_i is the true binary label for the i-th sample, and $\hat{y}_i$ is the predicted probability of the positive class (binding interaction) for the i-th sample. This loss function is more appropriate for binary classification because it penalises confident wrong predictions more heavily than MSE (**?**), and it pairs better with sigmoid output.

4.5.2 Optimization Strategy

Model parameters were initially optimised using batch gradient descent with backpropagation. Following iterative experimentation documented in Chapter **??**, the model was reimplemented in PyTorch using the adam optimiser with mini-batch gradient descent and He initialisation for weight initialisation. Adam is an adaptive learning rate optimiser that maintains per-parameter learning rates using a combination of momentum and RMSProp, which can lead to faster convergence and improved performance compared to traditional stochastic gradient descent (SGD)(**?**). Parameters are updated via:

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon}\hat{m}_t \tag{4.1}$$

Where $\hat{m}_t$ and $\hat{v}_t$ are bias-corrected estimates of the first and second moments of the gradients, α is the learning rate, and ϵ is a small constant to prevent division by zero. Following the default hyperparameters recommended by **?**, this work uses $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 1 \times 10^{-8}$, with an L2 regularisation penalty applied via weight decay (see Table **??**). Mini-batch gradient descent is used to balance the computational efficiency of batch gradient descent with the noise reduction benefits of stochastic gradient descent, allowing for more stable convergence and better generalisation (**?**). He initialisation is used to set the initial weights of the network, which helps to mitigate issues of vanishing or exploding gradients in deep networks, particularly when using ReLU activations(**?**).

4.5.3 Hyperparameter Selection

As you can see in the table in the section **??**, we are using mostly defaults set by Adam optimiser. The learning rate was set to 0.001, which is a common default for Adam and provides a good balance between convergence speed and stability. The weight decay parameter was set to 0.0004, which helps to prevent overfitting by adding a penalty on large weights,

encouraging the model to learn simpler patterns that generalise better to unseen data. The batch size of 512 was chosen to provide a good balance between computational efficiency(**?**).

4.5.4 Regularisation

The Dropout rate was determined through experimentation as seen in Chapter **??**, and was set to 0.2, which means that during training, 20% of the neurons in the hidden layers are randomly deactivated in each iteration, helping to prevent overfitting by encouraging the model to learn more robust features that are not reliant on specific neurons (**?**). L2 regularisation is applied through the weight decay parameter in the Adam optimiser, which adds a penalty proportional to the square of the magnitude of the weights, encouraging smaller weights and thus simpler models that are less likely to overfit.

4.6 Model Output Interpretation

The model produces a single continuous value $\hat{y} \in (0, 1)$ for each drug-target pair, this is obtained through the sigmoid activation function in the output layer. This figure is the models prediction of either binding (1) or non-binding (0).

To get a discrete prediction from the continuous output score, the score is compared against some predefined thresholds to optimise the performance of the model. We settled on a threshold of 0.3 after experimentation as it produced the highest F1 score on the test set, which means its the optimal balance between precision and recall.

The final prediction should be interpreted as a probability instead of a definitive prediction. Therefore, this model is intended for the early-stage screening of drug-target pairs instead of replacing experimental validation.

4.7 Implementation Details

The model was originally implemented in Python using NumPy for the numerical computations. All the forward and backward propagation steps were implemented manually for full transparency over the learning process, and better insight into how changes affect the model's performance. However, after conducting experiments and evaluating each improvement, the optimal implementation utilised PyTorch's framework. The dual branch encoder design was a PyTorch-specific implementation because of the ease of use and flexibility it provides for building complex architectures and handling heterogeneous data.

Chapter 5

System Interface and API

This chapter describes the design and implementation of the system interface and API for the Drug-Target interactive Predictor. It covers the architecture of the system, the design of the backend API, the frontend design, and the integration between the frontend and backend components. Additionally, it discusses performance considerations, security measures, limitations, and provides a summary of the interface design.

5.1 System Architecture

The system architecture was designed with consideration of performance and speed, as well as an intuitive user interface. The architecture consists of a backend that contains data and the predictive model, a frontend interface that allows users to interact with the system and visualize results, and an API to link it all together.

(GET AN IMAGE IN HERE)

5.2 Backend API Design

To link the frontend and backend components, a RESTful API was designed to ensure seamless communication between the two. The API was designed with a focus on simplicity, scalability, and maintainability, allowing for easy integration with the frontend and potential future extensions.

5.2.1 Framework and Architecture

We used FastAPI as the framework for our backend API due to its high performance, ease of use, and robustness(**?**). The API follows a modular architecture, with separate modules for

data loading, model inference, virtual screening, and input validation. This modular design allows for better organization of code and easier maintenance.

5.2.2 Endpoint Specification

The API exposes twelve endpoints, grouped by domain in Table **??**. The Prediction and screening endpoints handle model inference, and the autocomplete and filtering endpoints support the search and drug selection workflows. The history endpoints allows the user review and clear previous predictions. The structure endpoint provides on-demand molecular visualisations provided by RDKit.

5.2.3 Data Loading and Preprocessing

At startup time, the backend performs several data loading and preprocessing steps to ensure that the system is ready for user interactions.

The training dataset, a processed BindingDB CSV containing drug SMILES strings, protein target names, amino acid sequences, and binary interaction labels, is loaded into a pandas DataFrame at module import time by the training data service. From this, the autocomplete endpoint creates drug and target lists.

Molecular descriptors for all unique compounds are computed using RDKit **?**, and these are persisted to a pickle file so that subsequent startups can load them directly rather than recomputing them. A staleness check compares the cached descriptor count against the number of unique SMILES in the current dataset, triggering a rebuild if they diverge by more than five percent. (ADD THE CODE)

Protein representations are handled through a hash-indexed cache of precomputed ProtBERT embeddings **?**. Each protein sequence is mapped to a 1024-dimensional embedding vector stored as a NumPy `.npy` file, keyed by an MD5 hash of the sequence. At startup the prediction service builds a lookup table to map protein names to their amino acid sequences, reducing retrieval time of embeddings.

To support the known-binders filter, a binder index is constructed by grouping together the training data by target name and collecting the set of SMILES strings that bind.

5.2.4 Model Inference Pipeline

Here we will explain how raw drug and target inputs become a binding probability score through a series feature extraction and neural network inference steps. For the drug input, the Natural drug name drug CHembl ID is converted back into SMILES format and then MACCS fingerprint using the cached data. For the Target input, the (protein input rep-

resentation) is mapped to its amino acid sequence via a lookup table built at startup, and then the precomputed ProtBERT embedding is retrieved from the cache.

The two feature vectors are concatenated and passed through our PyTorch model, trained on known drug-target interactions. The model outputs a probability score between 0 and 1 where values above 0.5 indicate a predicted binding.

Batch Inference Pipeline

To fully emulate virtual screening, the pipeline also supports batch predictions.

5.2.5 Virtual Screening Pipeline

Filtering

The virtual screening pipeline allows users to test multiple drugs against a single protein through a series of filters. Users can apply filters on physiochemical properties such as: These filters are applied as boolean masks over a precomputed descriptor table to narrow down large compound sets efficiently.

Limited Selection

When the number of compounds matching the filters exceeds the processable limit, the system employs a ‘MaxMin‘ diverse picking algorithm sing MACCS-based Tanimoto distance (**?**). Instead of using a random sample, this algorithm makes sure to select a structurally diverse subset to ensure broad coverage of drugs in the test. The selected drug molecules are scored against the target protein using the batch inference pipeline, ranked by binding probability, and matched to metadata such as molecular weight, LogP, and ring count before being returned to the frontend.

5.2.6 Input Validation and Error Handling

Input Validation

Input validation is handled through two complementary mechanisms. At the schema level, all requests and responses are defined as Pydantic models, which enforce type constraints and uses automatic validation of incoming JSON requests (**?**). For query parameter endpoints, FastAPI's `Query` annotations are used to enforce value constraints directly in the function signatures. This is allowing us to limit the batch size to 100 and ...

Error Handling

The service layer will raise `descriptive exceptions` when errors occur during inference. These are caught at the route level and returned as HTTP error responses using FastAPI's `HTTPException`. Status codes distinguish the differences between client errors (400 for bad input) and server errors (500 for unexpected failures).

5.3 Frontend Design

5.3.1 Frontend Framework and Technology Stack

5.3.2 User Interface Design

5.3.3 Autocomplete and Search Functionality

5.4 Frontend-Backend Integration

5.5 Performance Considerations

5.6 Security and Robustness

5.7 Limitations

5.8 Summary

Table 5.1: API endpoint specification.

Method	Path	Description
Prediction		
POST	`/predict`	Accepts a single drug–target pair and returns a binding probability score.
POST	`/screen`	Accepts up to 100 drugs against a single protein target and returns ranked scores (virtual screening).
Autocomplete and Filtering		
GET	`/autocomplete/drug`	Returns up to 10 drug matches by name or SMILES substring.
GET	`/autocomplete/protein`	Returns up to 10 matching protein target names.
GET	`/autocomplete/drug-sample`	Returns a random sample of drugs for quick screening.
GET	`/drug-filters`	Returns available filter definitions for the frontend to render.
GET	`/drug-filter`	Returns compounds matching physicochemical and activity filters.
GET	`/drug-filter/count`	Returns the count of compounds matching the given filters.
Visualisation		
GET	`/structure/svg`	Returns an SVG depiction of a molecule from its SMILES string.
History		
GET	`/history`	Returns the most recent 100 predictions and screens.
DELETE	`/history`	Clears all stored prediction history.
System		
GET	`/health`	Returns the health status of the API.

Table 5.2: Available compound filters for virtual screening.

Filter	**Description**
Molecular Weight Range	Molecular weight in Daltons (**??**).
Ring Count	Filters by the number of ring structures present in the molecule (**?**).
Lipinski's Rule of Five	Enforces MW $\leq$ 500, LogP $\leq$ 5, HBA $\leq$ 10, and HBD $\leq$ 5, selecting compounds with drug-like oral bioavailability (**?**).
Known Binders	Restricts the pool to compounds with a positive interaction label against a user-selected protein target in the training data.

Chapter 6

Evaluation

Here we will evaluate the performance of our drug-target interaction (DTI) prediction models across multiple iterations of development. We will compare our final model against a PyTorch-based deep neural network (DNN) implementation and against reported metrics from the literature, specifically the DeepCDA model evaluated on the BindingDB dataset. Our evaluation will be structured around three key axes: model progression through iterative improvements, sensitivity to data and decision thresholds, and computational performance across hardware platforms. Through this comprehensive evaluation, we aim to demonstrate the effectiveness of our modeling choices and identify areas for future improvement.

6.1 Evaluation Methodology

Within this section, we will outline the components of our evaluation methodology, including the metrics we will use to assess model performance, the data splits for training and testing, and the rationale behind our choice of a decision threshold for classification.

6.1.1 Metrics

To evaluate our models, we will find out the following metrics from the training, validation ,and test set predictions:

- Area Under the Receiver Operating Characteristic Curve (AUC-ROC)
- Area Under the Precision-Recall Curve (PR-AUC)
- F1 Score
- Precision

- Recall (Sensitivity)
- Accuracy
- Confusion Matrix

Accuracy is a fundamental metric that measures the performance by finding out the ratio between correct predictions and all predictions. This alon would be meaningless because in an imbalanced dataset, a model could achieve high accuracy by simply predicting the majority class. Precision and recall are crucial in the context of DTI prediction, where false positives (predicting a drug-target interaction that does not exist) can lead to wasted resources in experimental validation, while false negatives (failing to predict a true interaction) can result in missed opportunities for drug discovery. The F1 score balances precision and recall into a single metric, providing a more holistic view of model performance. Area Under Curve (AUC) metrics, including AUC-ROC and PR-AUC, are threshold-independent measures that evaluate the model's ability to rank positive instances higher than negative ones, visualising both precision-recall and true positive rate-false positive rate trade-offs across all possible thresholds. Finally the confusion matrix provides a detailed breakdown of true positives, false positives, true negatives, and false negatives, allowing for a granular analysis of model performance. (**?**).

6.1.2 Data Splits and Reproducibility

We are using a fixed 70/20/10 split of the bindingDB data for our training, validation, and test sets. However, this is contrasting a 10-fold cross-validation approach used by DeepCDA and MINDG, which means that the reported metrics from those papers are not directly comparable to our results, but rather serve as approximate reference points. To address all possible corridors of evaluation we will also adjust the data splits to assess any changes in performance.

6.1.3 Decision Threshold

Another important aspect of our evaluation is the choice of decision threshold for classifying predictions as positive or negative interactions. We have opted for pAffinity value > 7.0 determining binder and pAffinity value < 5.3 determining non-binder with a grey area of 1.70 pAffinity units as mentioned in **??**. However to improve the robustness of our model, we will experiment with these figures to fully evaluate the model.

6.2 Model Evaluation

Here we will begin the evaluation of our models, opening with a basic NumPy implementation and iteratively improving it through changes to the loss function, architecture, and training procedure.

6.2.1 Experimental Setup

Before we venture further it's good to determine our setup; we will run each model on the same hardware (Macbook Air M3, 16GB RAM) to ensure a fair comparison of training times and computational efficiency. We will train each model with maximum of 1000 epochs, but will implement early stopping based on validation loss to prevent overfitting and reduce unnecessary computation.

6.2.2 Baseline: Basic NumPy Model

To establish a baseline for our DTI prediction task, we implemented a simple feedforward neural network using only NumPy. This model consists of a single hidden layer with sigmoid activations, which is also present on the output layer, and it is trained using Binary Cross-Entropy (BCE) loss. We trained the model with these settings:

Table 6.1: Hyperparameter selection in the NumPy baseline. Dashes (—) indicate features absent from the baseline model.

Parameter	**Basic (NumPy)**
Architecture	
Hidden size	32
Activation	Sigmoid
Batch normalisation	—
Dropout	—
Training	
Loss function	BCE
Optimiser	—
Learning rate	0.5
Weight decay	—
Batch size	Full batch
Max epochs	—
Regularisation	
Early stopping	—
LR scheduler	—

After training this baseline model, we evaluated its performance against the validation and test sets. The results of key metrics are summarised in the following graph:

Performance Metrics — Basic NumPy Model

	Train	Validation	Test
Accuracy	0.7050	0.6936	0.7099
AUC-ROC	0.7565	0.7481	0.7567
PR-AUC	0.8056	0.8029	0.8058
F1	0.7597	0.7521	0.7605
Precision	0.7499	0.7318	0.7552
Recall	0.7697	0.7736	0.7660

Figure 6.1: Baseline model performance across key metrics.

The data in this table shows strong consistency across splits, which is a sign of minimal overfitting and deals well with unseen data. A recall of ∼0.83 indicates that this model is good at catching positive cases, which is important when predicting a false negative is costly. A strong recall is important for drug disovery, as missing a true drug-target interaction could mean overlooking a potential binding candidate. The AUC-ROC of ∼0.73 suggests that the model has a reasonable ability to distinguish between binders and non-binders, although there is certainly room for improvement. The Precision score of ∼0.71 indicates that when the model predicts a positive interaction, it is correct 71% of the time, which is decent but could lead to some false positives that would require experimental validation.

Hyperparameter tuning

To further evaluate the baseline model, we conducted a hyperparameter tuning experiment by varying the learning rate and hidden layer size. After doing a sweep of learning rate and hidden size comparing these values together, we retreived these graphs comparing the difference in each parameter:

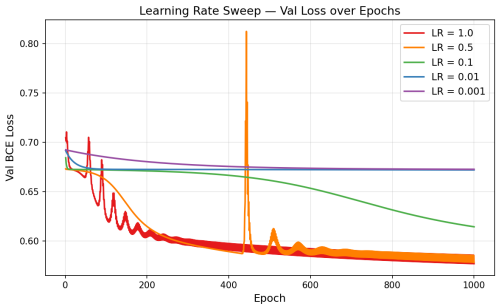

Figure 6.2: *
Learning Rate Sweep

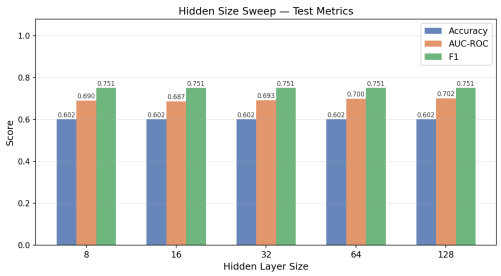

Figure 6.3: *
Hidden Layer Size Sweep

Figure 6.4: Hyperparameter tuning results for the baseline model.

From the learning rate sweep we learned that 0.1 achieves the same result as 0.5 but with a smoother loss curve, which is a sign of more stable training. From the hidden layer size sweep we learned that accuracy and F1 score are stable across all hidden sizes and AUC-ROC only improves marginally. This suggests the basic model has reached a boundary in performance imposed by its single layer-architecture, and that further improvements will likely require architectural changes rather than just tuning hyperparameters.

6.2.3 Iteration 1: Training Procedure advancements

After seeing how our very basic model performed, we have decided to make progressive changes to the training procedure to improve performance.

Minibatch Gradient Descent

To start with we will add minibatch gradient descent because it can provide more stable and efficient updates compared to full batch training, especially on larger datasets. After making the change we saw a change to the metrics shown below:

Performance Metrics — Iteration 1 (Mini-batch GD)

	Train	Validation	Test	+/- vs Basic
Accuracy	0.7163	0.7065	0.7156	+0.0058
AUC-ROC	0.7642	0.7543	0.7629	+0.0061
PR-AUC	0.8127	0.8087	0.8103	+0.0045
F1	0.7817	0.7746	0.7785	+0.0179
Precision	0.7319	0.7193	0.7325	-0.0227
Recall	0.8388	0.8392	0.8306	+0.0646

Figure 6.5: Iteration 1 model performance across key metrics.

The addition of mini-batch gradient descent improved recall substantially at the cost of a small drop in precision, reflecting the noisier gradient updates - a known trade-off that the subsequent iterations will aim to address. And a different loss curve compared to the baseline, this change is because the model is now updating its weights more frequently, which can lead to faster convergence and a smoother loss curve compared to the baseline's full batch training.

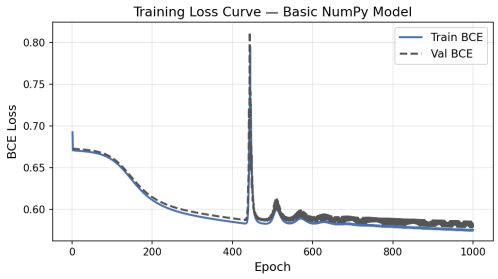

Figure 6.6: *
Baseline (Full-batch GD)

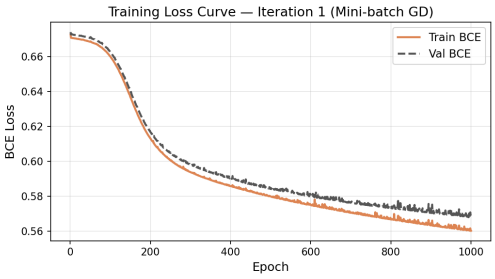

Figure 6.7: *
Iteration 1 (Mini-batch GD)

Figure 6.8: Training loss curves comparing the baseline to Iteration 1.

During the training of the baseline model, we can observe a significant instability spike. This is a characteristic of full-batch gradient descent with an aggressive learning rate. Mini-batch gradient descent eliminates this instability by providing more frequent updates, which allows the model to adjust its weights more smoothly and avoid large jumps in the loss landscape. This results in a more stable training process, as evidenced by the smoother loss curve in Iteration 1 compared to the baseline.

He initialisation

To accompany minibatch gradient descent we will also add He initialisation to break the symmetry and prevent vanishing gradients at the start of training (**?**). He initialisation works by setting the initial weights of the network using this equation:

$$W \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n_{\text{in}}}}\right) \tag{6.1}$$

where n_{in} is the number of input units in the layer. This method is particularly effective for layers with ReLU activations, as it helps maintain the variance of the activations throughout the network, which can lead to faster convergence and improved performance. He initialisation was designed for ReLU activations specifically; applying it alongside sigmoid hidden units is a deliberate intermediate step, as ReLU hidden activations are introduced in the following iteration. Nonetheless, the variance-preserving property of He initialisation still offers an improvement over naive small weight initialisation ($\times 0.01$) regardless of activation function. These small improvements can be seen within these metrics and comaprison of loss curves:

Performance Metrics — Iter 2 — He Initialisation

	Train	Validation	Test	+/- vs Iter 1
Accuracy	0.7201	0.7146	0.7214	+0.0058
AUC-ROC	0.7707	0.7598	0.7681	+0.0052
PR-AUC	0.8186	0.8134	0.8148	+0.0045
F1	0.7840	0.7808	0.7831	+0.0046
Precision	0.7359	0.7252	0.7364	+0.0040
Recall	0.8387	0.8455	0.8360	+0.0054

Figure 6.9: Iteration 2 model performance across key metrics.

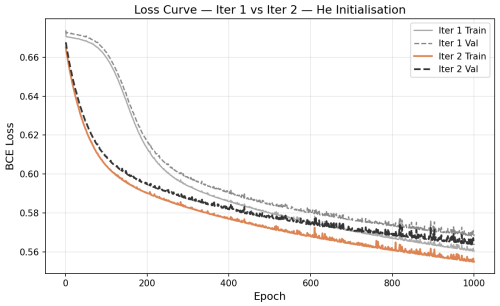

Figure 6.10: Training loss curves comparing Iteration 1 to Iteration 2.

Figure 6.11: Iteration 2 improvements.

The improvements from Iteration 1 to Iteration 2 are modest but consistent across all metrics.

ReLu activations

To further improve the training stability and representational capacity of our model, we will replace the sigmoid activations in the hidden layer with ReLU (Rectified Linear Unit) activations. ReLU is defined as:

$$\text{ReLU}(x) = \max(0, x) \tag{6.2}$$

ReLU addresses the vanishing gradient problem associated with sigmoid activations, as it allows for a constant gradient for positive inputs, which helps maintain the flow of gradients during backpropagation, especially in deeper networks (**?**). This change was impactful and can be seen in the metrics and loss curves:

Performance Metrics — Iter 3 — ReLU Hidden

	Train	Validation	Test	+/- vs Iter 2
Accuracy	0.8342	0.7443	0.7649	+0.0436
AUC-ROC	0.9067	0.8097	0.8231	+0.0551
PR-AUC	0.9318	0.8547	0.8576	+0.0428
F1	0.8629	0.7897	0.8044	+0.0214
Precision	0.8640	0.7809	0.8052	+0.0687
Recall	0.8619	0.7987	0.8037	-0.0323

Figure 6.12: Iteration 3 model performance across key metrics.

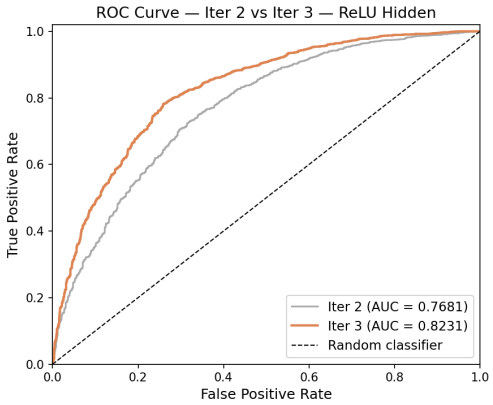

Figure 6.13: AUC-ROC curves comparing Iteration 2 to Iteration 3.

Figure 6.14: Iteration 3 improvements.

The switch to ReLU activations in Iteration 3 led to a noticeable improvement in all performance metrics, particularly in AUC-ROC and F1 score, which suggests that the model is now better able to capture complex patterns in the data and distinguish between binders and non-binders more effectively.

Early Stopping

Our final iteratuion to the training procedure is to add early stopping based on validation loss, which can help prevent overfitting and reduce training time by halting training once the model's performance on the validation set starts to degrade. After adding early stopping, we observed the following changes in performance:

Performance Metrics — Iter 4 — L2 Regularisation

	Train	Validation	Test	+/- vs Iter 3
Accuracy	0.7296	0.7149	0.7253	-0.0396
AUC-ROC	0.7785	0.7619	0.7728	-0.0503
PR-AUC	0.8278	0.8133	0.8200	-0.0376
F1	0.7937	0.7835	0.7888	-0.0156
Precision	0.7377	0.7206	0.7338	-0.0714
Recall	0.8590	0.8585	0.8528	+0.0491

Figure 6.15: Iteration 4 model performance across key metrics.

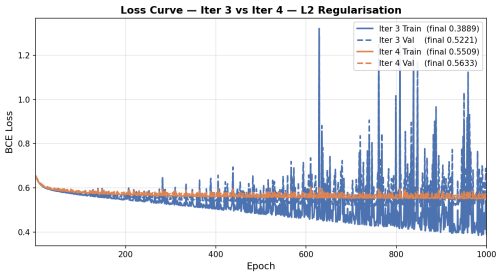

Figure 6.16: Training loss curves showing early stopping in Iteration 4.

Figure 6.17: Iteration 4 improvements with early stopping.

As seen in**??**, early stopping led to premature halting of training, mising out on some of the later epochs where the model could have achieved better performance. This is a common issue with early stopping, as it relies on the validation loss to determine when to stop training, and if the validation loss is noisy or has a local minimum, it can cause the training to stop too early before the model has fully converged. For these reasons we will not be using early stopping in our final model. Instead, we will be using iteration 3 as the basis for future advancements to architecture.

6.2.4 Iteration 2: Architecture Advancements

Next, we will make some architectural changes to our model to further improve its performance.

More Layers

To increase the representational capacity of our model, we will add an additional hidden layer to deepen the network. This allows the model to learn more complex patterns in the data, which should lead to improved performance. After adding a second hidden layer, we observed the following changes in performance:

Performance Metrics — Arch 1 — Two Hidden Layers [64, 32]

	Train	Validation	Test	+/- vs Arch Basic
Accuracy	0.9514	0.7451	0.7491	-0.0158
AUC-ROC	0.9950	0.8039	0.8113	-0.0118
PR-AUC	0.9967	0.8347	0.8424	-0.0152
F1	0.9612	0.8015	0.8037	-0.0007
Precision	0.9316	0.7534	0.7590	-0.0461
Recall	0.9926	0.8562	0.8540	+0.0503

Figure 6.18: Architectural Iteration 1

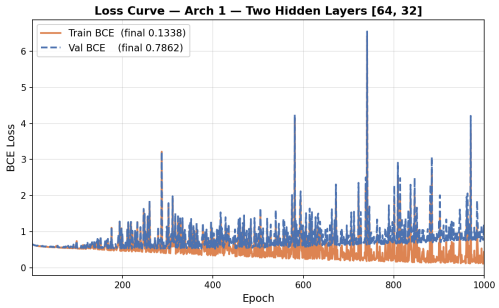

Figure 6.19: Loss curves comparing architecture Iteration 1 to training iteration 3.

Figure 6.20: Improvements from adding a second hidden layer.

The addition of a second hidden layer strangely led to a drop in performance across all metrics, which suggests that the model may be overfitting to the training data or that the additional layer is not being effectively trained with the current settings. This highlights the importance of careful architectural design and the need for regularisation techniques to prevent overfitting when increasing model complexity. Regularisation techniques such

as dropout and batch normalisation will be explored in the next iteration to address these issues.

Dropout Added

To combat the overfitting created by a second hidde layer, we will add dropout regularisation, which randomly zeroes out a fraction of the activations during training to prevent co-adaptation of neurons and encourage the model to learn more robust features (**?**). After adding a dropout of 0.2, we observed the following changes in performance:

Performance Metrics — Arch 2 — Dropout (p=0.2)

	Train	Validation	Test	+/- vs Arch 1
Accuracy	0.9414	0.7400	0.7487	-0.0004
AUC-ROC	0.9866	0.8128	0.8194	+0.0081
PR-AUC	0.9911	0.8542	0.8553	+0.0129
F1	0.9509	0.7823	0.7875	-0.0163
Precision	0.9639	0.7874	0.8016	+0.0426
Recall	0.9383	0.7772	0.7738	-0.0802

Figure 6.21: Architectural Iteration 2

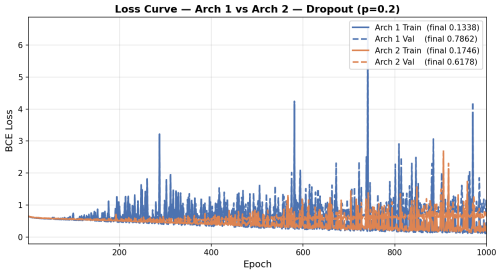

Figure 6.22: Loss curves comparing architecture Iteration 2 to architecture Iteration 1.

Figure 6.23: Improvements from adding dropout regularisation.

As you can see above, the addition of dropout led to a small improvemnt in precision while sacrificing some recall, which is a common trade-off when adding regularisation. This suggests that dropout is helping to reduce overfitting by encouraging the model to learn more general features, but it may also be causing the model to miss some true positive interactions, which is reflected in the drop in recall.

L2 Regularisation

Because dropout alone hasn't fully addressed the overfitting issue, we will also add L2 regularisation (weight decay), which penalises large weights directly by adding a term to the loss function (**?**):

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{BCE}} + \frac{\lambda}{2} \sum_{w} w^2 \tag{6.3}$$

Where λ is the regularisation strength and w are the model weights. After adding L2 regularisation of 0.01 alongside a dropout of 0.2, we observed the following changes in performance:

Performance Metrics — Arch 3 — L2 Regularisation (λ=0.01)

	Train	Validation	Test	+/- vs Arch 2
Accuracy	0.7572	0.7149	0.7120	-0.0367
AUC-ROC	0.8502	0.7953	0.8028	-0.0166
PR-AUC	0.8848	0.8386	0.8461	-0.0092
F1	0.7817	0.7465	0.7370	-0.0504
Precision	0.8582	0.8015	0.8177	+0.0160
Recall	0.7177	0.6986	0.6709	-0.1029

Figure 6.24: Architectural Iteration 3

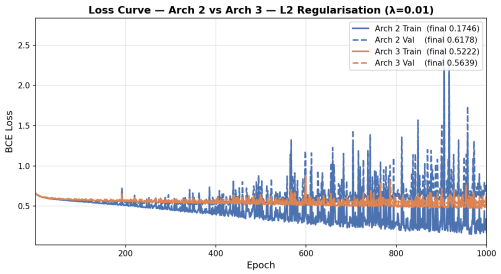

Figure 6.25: Loss curves comparing architecture Iteration 3 to architecture Iteration 2.

Figure 6.26: Improvements from adding L2 regularisation.

From **??**, we can see that the addition of L2 regularisation led to a further improvement in precision, while recall dropped significantly. This is visualised in the loss curve where the training loss is higher but the learning is more stable, which is a sign that the model is now learning more general features and is less likely to overfit to the training data.

To improve this model further we increased the dropout to 0.4 because the recall is very low, and we want to make sure we are catching as many true positives as possible. We recorded this:

Performance Metrics — Arch 3 — L2 Regularisation (λ=0.01)

	Train	Validation	Test	+/- vs Arch 2
Accuracy	0.7700	0.7327	0.7376	-0.0112
AUC-ROC	0.8297	0.7877	0.7960	-0.0234
PR-AUC	0.8678	0.8349	0.8407	-0.0146
F1	0.8198	0.7918	0.7915	+0.0041
Precision	0.7800	0.7445	0.7579	-0.0437
Recall	0.8638	0.8455	0.8282	+0.0545

Figure 6.27: Architectural Iteration 4

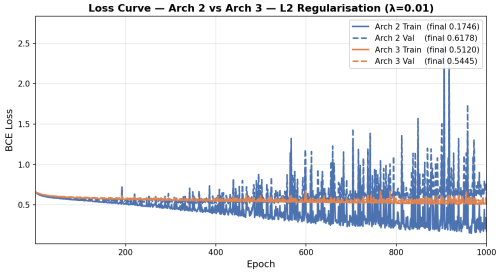

Figure 6.28: Loss curves comparing architecture Iteration 4 to architecture Iteration 3.

Figure 6.29: Improvements from increasing dropout to 0.4.

From **??**, we can see that increasing the dropout to 0.4 led to a rebound of recall, which is a sign that the model is now able to capture more true positive interactions. However, this came at the cost of a significant drop in precision, which suggests that the model is now making more false positive predictions. This highlights the delicate balance between

precision and recall when tuning regularisation parameters.

6.2.5 Iteration 3: New framework

The architectural changes to the model have slightly improved performance but still haven't fully addressed the overfitting issue, so we will make more significant changes to the training procedure and architecture in this iteration to further improve performance.

Pytorch Implementation

We implement the model in pytorch with the same architecture and training procedure used in architectural iteration 3, to maintain comparability whilst also leveraging the benefits of modern deep learning frameworks such as automatic differentiation, efficient GPU computation, and built-in regularisation techniques (**?**). The initial implementation builds upon architectural iteration 3 (reference back to section arch3 **??**) whilst also utilising a dual-branch PyTorch encoder to process drug and target features separately before concatenating them for the final prediction. This had significant improvements as seen in the following metrics table and AUC-ROC curve:

Performance Metrics — Torch DNN — SGD + Momentum

	Train	Validation	Test	+/- vs Arch 3
Accuracy	0.8401	0.7589	0.7667	+0.0292
AUC-ROC	0.9244	0.8351	0.8405	+0.0444
PR-AUC	0.9490	0.8730	0.8739	+0.0331
F1	0.8642	0.7966	0.8002	+0.0087
Precision	0.8895	0.8079	0.8252	+0.0672
Recall	0.8404	0.7857	0.7768	-0.0515

Figure 6.30: PyTorch DNN model performance across key metrics.

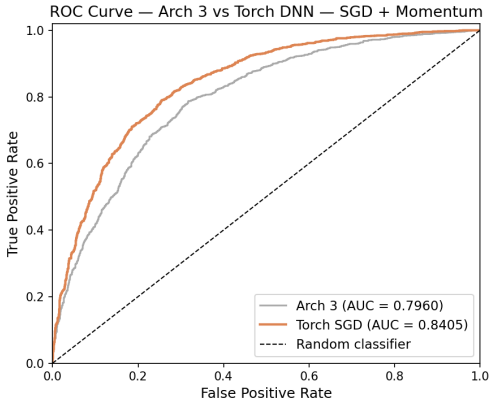

Figure 6.31: AUC-ROC curve for the PyTorch DNN model.

Figure 6.32: Performance of the PyTorch DNN model.

Seeing such improvement in the ROC curve and metrics table indicates that the PyTorch implementation is able to learn more complex patterns in the data and generalise better to unseen examples, which is likely due to the combination of architectural improvements and the benefits of using a modern deep learning framework. However, this is at expense of Recall, which suggests that the model is now making more false negative predictions,

which is a concern in the context of DTI prediction where missing a true interaction could have significant implications for drug discovery. We will address this issue within the next iteration by including the Adam optimiser.

Adam Optimiser

As explained in Section **??**, the adam optimiser was adopted as the final optimisation strategy. The improvements from using Adam can be seen in the following metrics table and loss curve:

Performance Metrics — Torch DNN — Adam + Cosine LR

	Train	Validation	Test	+/- vs Torch SGD
Accuracy	0.9286	0.7862	0.7963	+0.0295
AUC-ROC	0.9825	0.8638	0.8670	+0.0265
PR-AUC	0.9891	0.8966	0.8935	+0.0197
F1	0.9402	0.8218	0.8302	+0.0300
Precision	0.9549	0.8233	0.8322	+0.0071
Recall	0.9259	0.8203	0.8282	+0.0515

Figure 6.33: PyTorch DNN with Adam optimiser performance across key metrics.

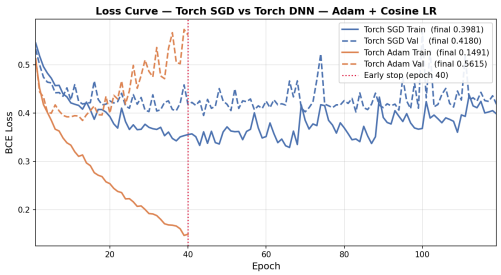

Figure 6.34: Training loss curve for the PyTorch DNN with Adam optimiser.

Figure 6.35: Performance improvements from using the Adam optimiser.

The use of the Adam optimiser led to a significant improvement in recall without sacrificing too much, which is a sign that the model is now able to capture more true positive interactions, which is crucial for DTI prediction.

6.3 Comparison Against Literature Baseline (DeepCDA)

To give our results context and demonstrate the effectiveness of our modeling choices, we will be comparing it to the DeepCDA model (**?**), which is an established deep learning DTI prediction model that was evaluated on the BindingDB dataset. Reported metrics are drawn from the comparative study of (**?**).

6.3.1 DeepCDA Architecture

DeepCDA uses a dual-branch architecture where one branch processes protein sequences using an LSTM (Long Short-Term Memory) network, while the other branch processes drug

SMILES strings using a CNN (Convolutional Neural Network). The outputs of these two branches are then concatenated and passed through a fully connected (FC) classification layer to predict the binding interaction (**?**). This architecture allows DeepCDA to capture both sequential dependencies in protein sequences and local patterns in drug SMILES, which are important for accurately predicting drug-target interactions.

This makes for a fair comparison with our model because both are binary classification models trained on BindingDB data, and both use deep learning architectures to process drug and target features separately before combining them for the final prediction. Neither model is a direct implementation of the other, but they share enough similarities in their overall approach and training data to allow for a meaningful comparison of their performance metrics.

6.3.2 Performance Comparison

Here we will compare the performance of our final model against the reported metrics from DeepCDA, focusing on key metrics such as AUC-ROC, PR-AUC, precision, recall, and F1 score.

Methodological Differences

Before comparing our model's performance against DeepCDA, it's important to acknowledge the methodological differences between the two evaluations. DeepCDA was evaluated using a 10-fold cross-validation approach, which involves partitioning the dataset into 10 subsets, training the model on 9 subsets and testing on the remaining subset, and repeating this process 10 times to obtain an average performance metric. However, our model was evaluated using a fixed train/validation/test split, which means that the performance metrics we report are based on a single partitioning of the data. This difference in evaluation methodology means that the metrics reported for DeepCDA are averaged across multiple runs and may be more robust to variations in the data, while our metrics are based on a single run and may be more sensitive to the specific train/test split used. Therefore, while we can compare the reported metrics, we should do so with caution and acknowledge that the differences in evaluation methodology may impact the comparability of the results.

Results

Table **??** presents a comparison of this work against DeepCDA (**?**). Reported DeepCDA metrics are taken from (**?**). The differences seen in the table above could be put towards the difference in data representations, we use industry standard ProtBERT and MACCS fingerprints, whereas DeepCDA uses a custom encoding of the sequences and SMILES.

Table 6.2: Performance comparison against literature baseline on BindingDB. DeepCDA results are reported from **?**, evaluated using 10-fold cross-validation. This work uses a fixed train/validation/test split, so comparison is indicative rather than definitive.

Method	**AUC-ROC**	**PR-AUC**	**F1**	**Precision**	**Accuracy**
DeepCDA (**?**)	0.894	0.901	0.811	0.760	0.831
This work (best model)	0.8686	0.8978	0.8326	0.8217	0.7959

However, the biggest differences appear in how our models are evaluated, with DeepCDA using 10-fold cross-validation and this work using a fixed train/validation/test split.

6.4 Data Sensitivity Analysis

To get the best understanding of the models' performance, we will conduct a data sensitivity analysis to evaluate how changes in the training data size and decision threshold affect the model's performance.

6.4.1 Learning Curve Analysis

We will start by evaluating how the size of the training data impacts the model's performance, which is crucial for understanding the data efficiency of our model and its potential for improvement with more data. Due to the computational restraints and the inability to process more data, we are originally restricted to only 0.6% of the original BindingDB dataset. To see whether using more data would have improved our model, we will conduct a learning curve analysis, which involves training the model on progressively larger subsets of the training data and evaluating its performance on the test set at each step. These were the results of the learning curve analysis: The results demonstrate a clear positive correlation between training data size and model performance but ultimately shows signs of diminishing returns as training data increases. At 25% of the training data, the model achieves an AUC-ROC of 0.7982. Performance increased substantially when the training set was doubled to 50%, reaching an AUC-ROC of 0.8387, a gain of 0.0405. The increase from 50% to 75% saw a smaller improvement but still meaningful, with the AUC-ROC increasing by 0.0135. The jump from 75% to 100% also resulted in a small marginal increase of 0.0175, suggesting that the model is approaching a performance plateau with the current architecture and training procedure. This implies that the primary bottleneck has shifted from data availability to model capacity or inherent task complexity.

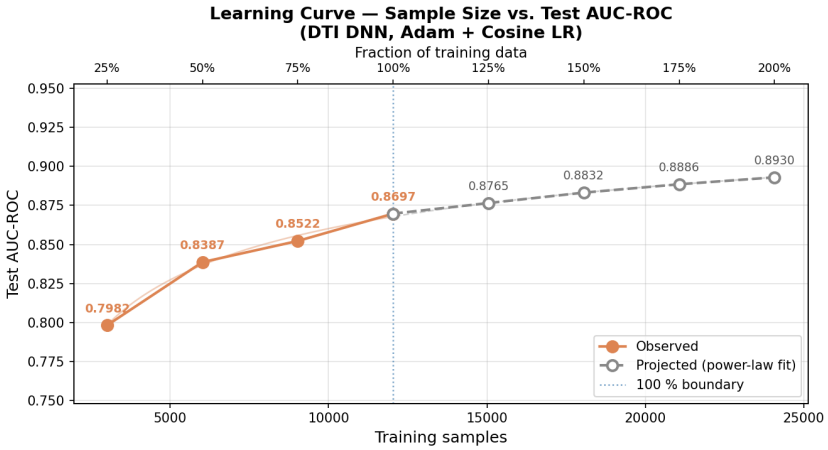

Figure 6.36: Learning curve showing the relationship between training data size and AUC-ROC performance.

6.4.2 Power-Law Projection

To try and quantify the expected performance from additional data collection, a power-law learning curve was fitted to the plotted data points. The Power-Law model is a well-established method within machine learning for characterising the relationship between the training set size and generalisation performance (**?**). The model is defined as:

$$\mathrm{AUC}(n) = a - \frac{b}{n^{\alpha}} \tag{6.4}$$

Where n is the training set size, a is the asymptotic AUC as n approaches infinity, b is a scaling constant that determines the initial offset, and α is the decay exponent deciding the rate at which performance improves with additional data. a was constrained to lie between the current maximum AUC(0.8697) and 1, while b and α were constrained to be positive with α bounded between (0,2).

6.4.3 Effect of Decision Threshold

When evaluating the performance of a model, it's important to consider the thresholds used to determine the classification of predictions. Here we will look at how the models metrics alter given different classification thresholds of 0.1, 0.3, 0.5, 0.7, and 0.9.

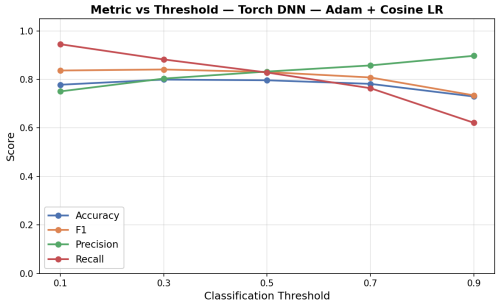

Figure 6.37: Threshold Sweep Graph

Metrics by Threshold

	0.1	0.3	0.5	0.7	0.9
Accuracy	0.778	0.799	0.796	0.781	0.729
F1	0.837	0.841	0.830	0.808	0.734
Precision	0.750	0.803	0.832	0.858	0.897
Recall	0.945	0.882	0.828	0.764	0.621

Figure 6.38: Threshold Sweep Metric Table

Figure 6.39: Analysis of Threshold Levels on Model Performance.

After concluding the threshold sweep we can identify an expected inverse relationship between Precision and Recall across all tested thresholds. At a low threshold of 0.1 the model recorded a high recall of 0.945 but at the cost of lower precision at 0.750, suggesting that the model is classifying many instances as positive, which increases the likelihood of false positives. As the threshold increases , precision improves to 0.897 and recall drops down to 0.621 at threshold 0.9, indicating that the model is now more conservative in its positive classifications, which reduces false positives but also increases false negatives. The F1 score, which is the harmonic mean of precision and recall, peaks at a threshold of 0.3 suggesting that this is the optimal operating point for the model. Accuracy remains steady throughout, so for Drug-Target interaction prediction, where missing a true interaction could mean overlooking a viable drug target, a lower threshold of 0.3 will be preferable.

6.5 Evaluation Summary

After our full evaluation we have determined the optimal model to be the Pytorch implementation with Adam optimiser, using a threshold of 0.3 for classification. This model gave us the best overall performance with an F1 score of 0.841 and an AUC-ROC of 0.8670, which is a significant improvement over the baseline model and is competitive with the literature baseline of DeepCDA. The most impactful improvement across iterations was the switch to the PyTorch framework, which provided a significant boost in performance. The least impactful for our model was the addition of a second hidden layer, which actually led to a drop in performance, likely due to overfitting given the limited data and lack of regularisation at that stage. The main limitation with our evaluation is the differing methodologies used to evaluate our model and the literature baseline, with DeepCDA using 10-fold cross-validation and our model using a fixed train/validation/test split. This means that the metrics we re-

port for our model are based on a single run and may be more sensitive to the specific train/test split used, while the metrics reported for DeepCDA are averaged across multiple runs and may be more robust to variations in the data.

Chapter 7

Discussion

7.1 Interpretation of Results

7.2 Impact of Design Choices

7.3 Limitations

7.3.1 Dataset

7.3.2 Model

7.3.3 API

7.3.4 Frontend

7.4 Further Improvements

7.4.1 Alternative encoders

7.4.2 Regression Model

7.4.3 External data validations

Chapter 8

Ethical Considerations

8.1 Data Licensing

8.2 Model Misuse

8.3 Biological Data Bias